

Name: P.Chithra

Hall no:2303A52137

Assignment-7.3

Task-1:

Fixing Syntax Errors

Scenario

You are reviewing a Python program where a basic function definition contains a syntax error.

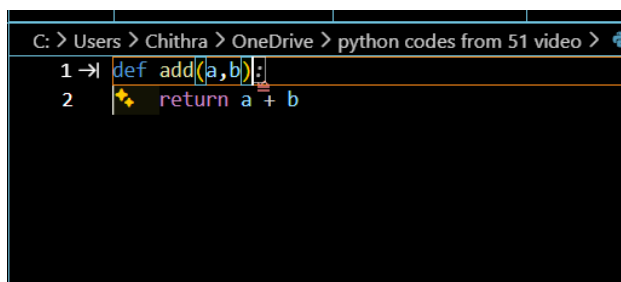
Requirements

- Provide a Python function `add(a, b)` with a missing colon
- Use an AI tool to detect the syntax error
- Allow AI to correct the function definition
- Observe how AI explains the syntax issue

Expected Output

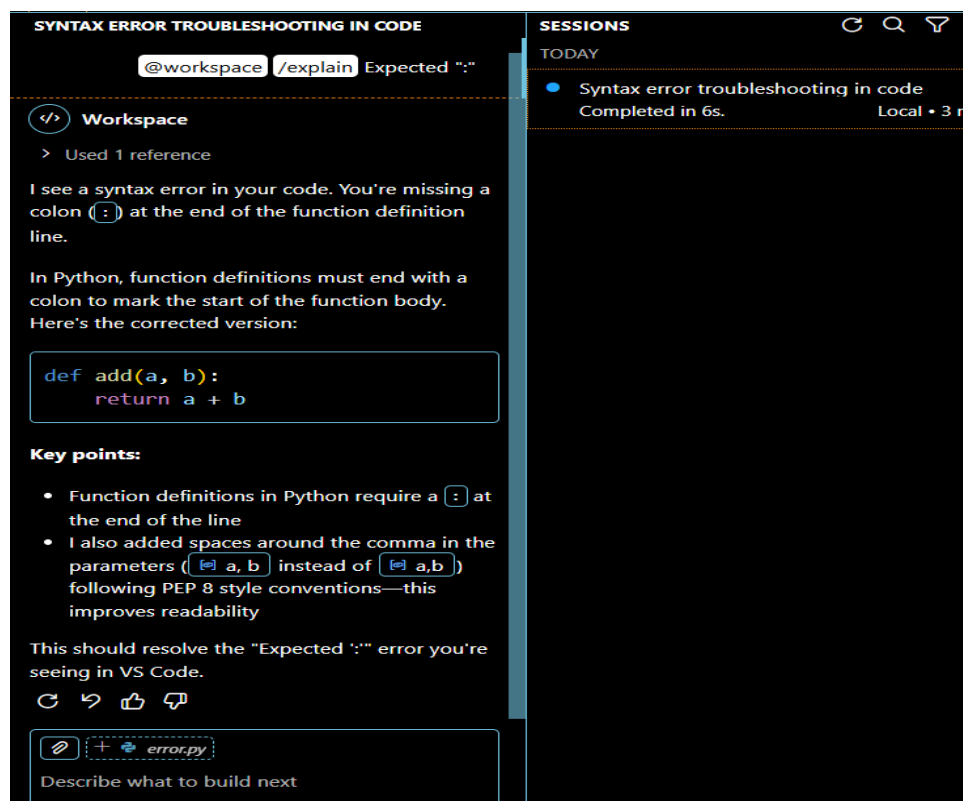
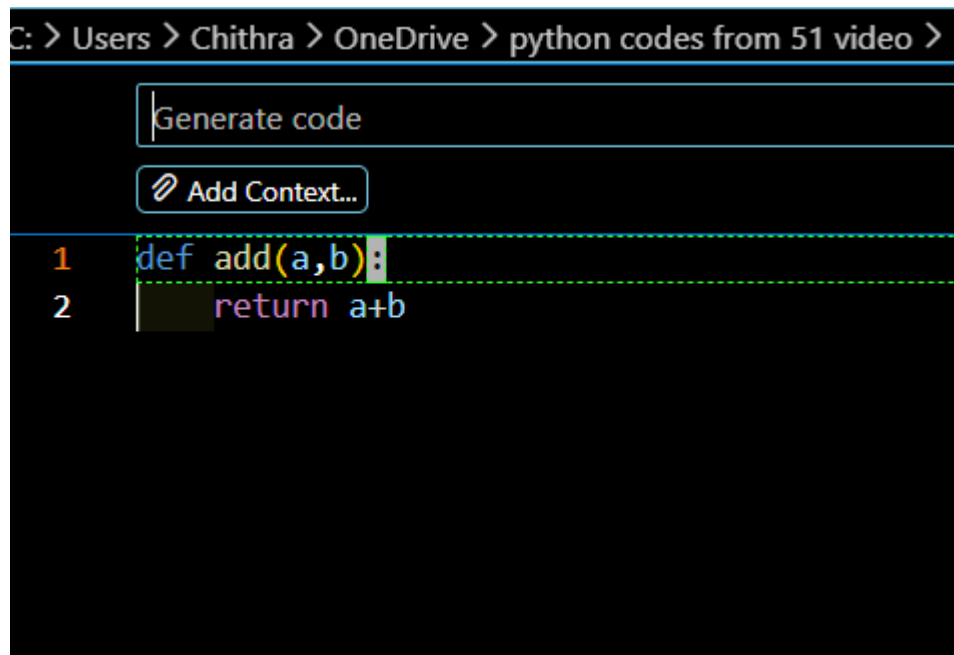
- Corrected function with proper syntax
- Syntax error resolved successfully
- AI-generated explanation of the fix

Code:



```
C: > Users > Chithra > OneDrive > python codes from 51 video >  
1 → def add(a,b):  
2   return a + b
```

The screenshot shows a code editor window with a file explorer path at the top: 'C: > Users > Chithra > OneDrive > python codes from 51 video >'. The code editor contains two lines of Python code. Line 1 is 'def add(a,b):' and line 2 is 'return a + b'. A red squiggly line is under the closing parenthesis of the function definition on line 1, indicating a syntax error (missing colon). A yellow star icon is visible next to line 2.



Explanation:

In Python, a function definition must end with a colon (:).

The colon tells Python that an indented block of code follows the function

header.

Without the colon, Python raises a `SyntaxError`.

Adding the colon after `def add(a, b)` fixes the issue and allows the function to run correctly.

Task-2:

Debugging Logic Errors in Loops

Scenario

You are debugging a loop that runs infinitely due to a logical mistake.

Requirements

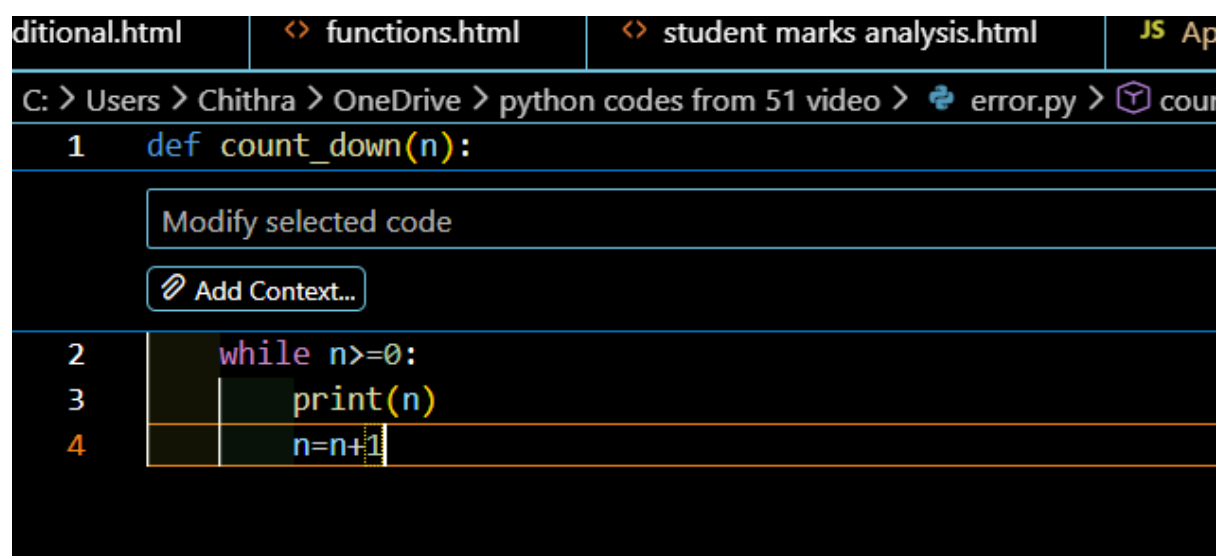
- Provide a loop with an increment or decrement error
- Use AI to identify the cause of infinite iteration
- Let AI fix the loop logic
- Analyze the corrected loop behavior

Expected Output

- Infinite loop issue resolved
- Correct increment/decrement logic applied
- AI explanation of the logic error

Code:

Modification



The screenshot shows a code editor with a dark theme. At the top, there are tabs for 'ditional.html', 'functions.html', 'student marks analysis.html', and 'JS Ap'. The active file is 'error.py', located at 'C: > Users > Chithra > OneDrive > python codes from 51 video > error.py'. The code in the editor is as follows:

```
1 def count_down(n):  
      
    while n>=0:  
        print(n)  
        n=n+1
```

Line 1 is highlighted in blue. A context menu is open over the code, showing options 'Modify selected code' and 'Add Context...'. The cursor is positioned at the end of line 4, which is `n=n+1`.

Review

```
C: > Users > Chithra > OneDrive > python codes from 51 video > error.py > count_down
1 def count_down(n):
2     while n>=0:
3         print(n)
4         n=n+1
```

Comment 1 of 1

Code Review

The statement `n=n+1` causes an infinite loop or counts up instead of down. To count down, you should decrement `n` with `n = n - 1` or `n -= 1`.

No change found to suggest.

Explanation:

The infinite loop occurred due to incorrect update logic.

The loop variable was moving in the opposite direction of the termination condition.

By incrementing `i` instead of decrementing it, the loop eventually violates the condition and stops executing.

Task 3:

Handling Runtime Errors (Division by Zero)

Scenario

A Python function crashes during execution due to a division by zero error.

Requirements

- Provide a function that performs division without validation
- Use AI to identify the runtime error
- Let AI add try-except blocks for safe execution
- Review AI's error-handling approach

Expected Output

- Function executes safely without crashing
- Division by zero handled using try-except
- Clear AI-generated explanation of runtime error handling

Code:modification

```
1 def divide(a,b):  
2     ...  
3     return a/b  
4     print(divide(10,0))
```

Review

Comment 1 of 2

Code Review

Division by zero is not handled. Calling `divide(10, 0)` will raise a `ZeroDivisionError`. Consider adding error handling to manage this case gracefully.

No change found to suggest.

Apply and Go to Next | Discard and Go to Next

Explanation:

The given program causes a runtime error because it attempts to divide a number by zero.

Division by zero is not allowed in Python, so a `ZeroDivisionError` occurs during execution.

This error is called a runtime error because it happens while the program is running, not during compilation.

To fix this, a try-except block is used to handle the error safely.

The try block contains the division operation, and the except block catches the error.

Instead of crashing, the program displays a meaningful error message.

This ensures safe execution and proper error handling.

Task 4:

Debugging Class Definition Errors

Scenario

You are given a faulty Python class where the constructor is incorrectly defined.

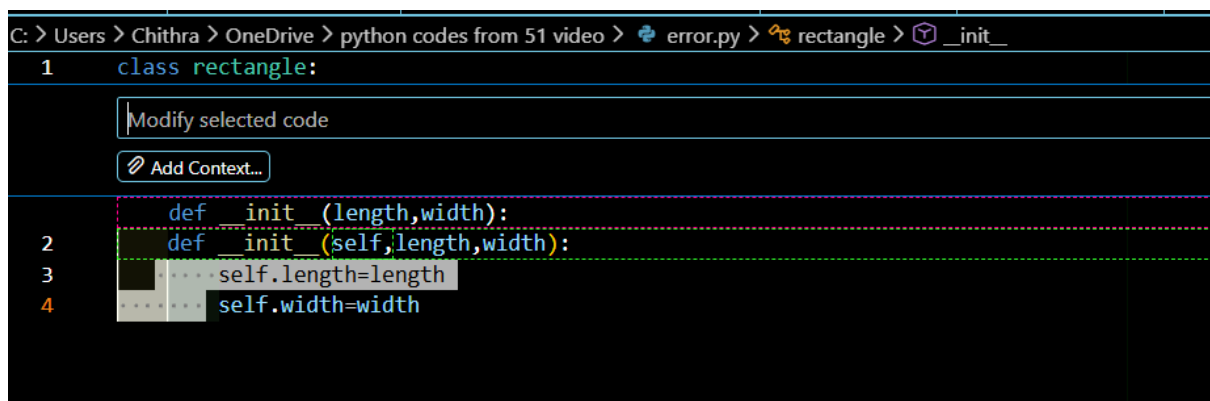
Requirements

- Provide a class definition with missing self-parameter
- Use AI to identify the issue in the `__init__()` method
- Allow AI to correct the class definition
- Understand why self is required

Expected Output

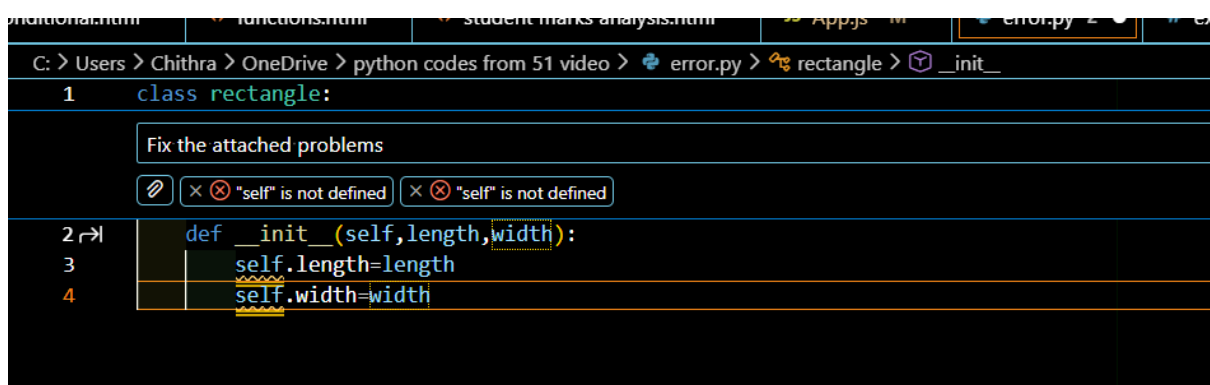
- Corrected `__init__()` method
- Proper use of self in class definition
- AI explanation of object-oriented error

Code:Modification



```
C: > Users > Chithra > OneDrive > python codes from 51 video > error.py > rectangle > __init__  
1 class rectangle:  
    def __init__(length,width):  
        self.length=length  
        self.width=width
```

Fixing



```
C: > Users > Chithra > OneDrive > python codes from 51 video > error.py > rectangle > __init__  
1 class rectangle:  
    def __init__(self,length,width):  
        self.length=length  
        self.width=width
```

Review

```
C: > Users > Chithra > OneDrive > python codes from 51 video > error.py > rectangle > __init__  
1 class rectangle:  
2     def __init__(length,width):  
  
Comment 1 of 2  
  
 Code Review  
The __init__ method is missing the self parameter as its first argument. It should be def __init__(self, length, width):.  
No change found to suggest.  
  
[Apply and Go to Next] [Discard and Go to Next]  
  
3     self.length=length  
4     self.width=width
```

Explanation:

The given class produces an error because the `__init__` constructor is missing the `self` parameter.

In Python, `self` refers to the current object of the class.

Without `self`, Python cannot assign values to instance variables.

This causes an error when an object of the class is created.

AI identifies that `__init__` must always take `self` as the first parameter.

By adding `self` to the constructor, the class works correctly.

This ensures proper object initialization and correct use of object-oriented principles.

Task 5:

Resolving Index Errors in Lists

Scenario

A program crashes when accessing an invalid index in a list.

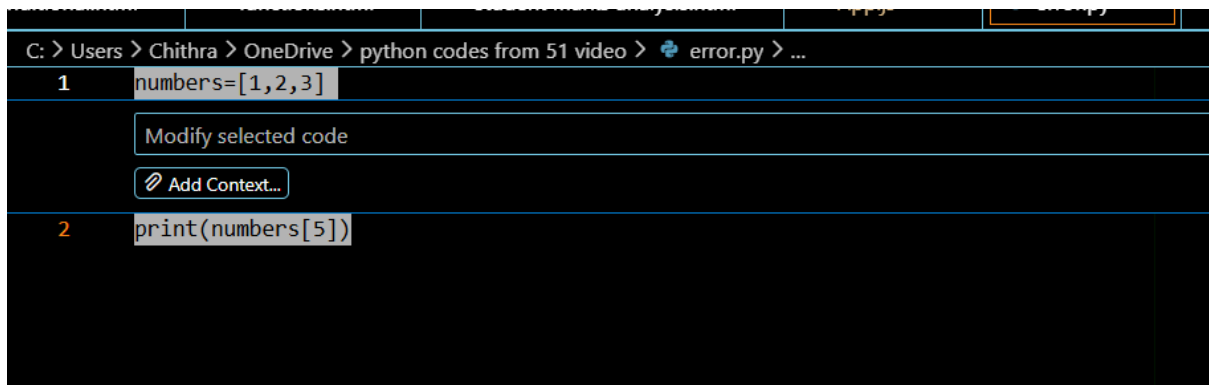
Requirements

- Provide code that accesses an out-of-range list index
- Use AI to identify the Index Error
- Let AI suggest safe access methods
- Apply bounds checking or exception handling

Expected Output

- Index error resolved
- Safe list access logic implemented

Modification

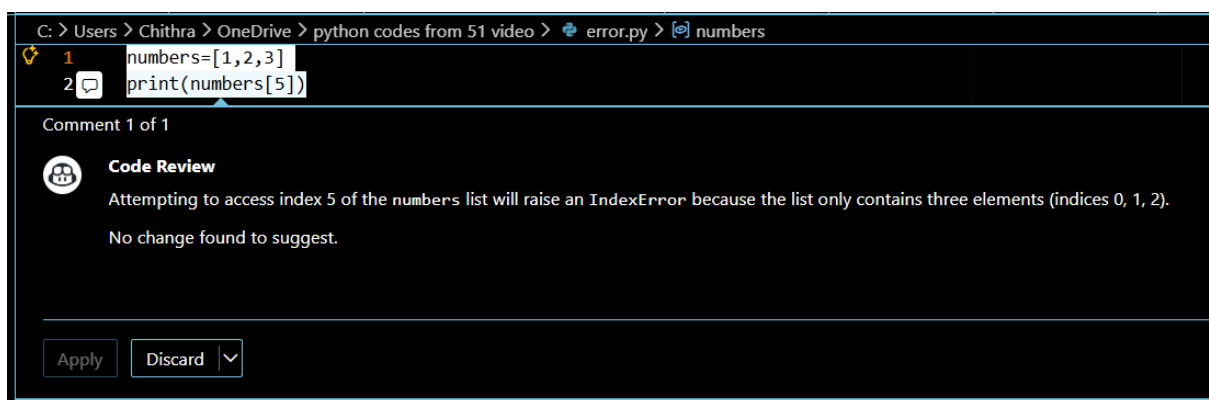


```
C: > Users > Chithra > OneDrive > python codes from 51 video > error.py > ...  
1 numbers=[1,2,3]  
2 print(numbers[5])
```

Modify selected code

Add Context...

Review



```
C: > Users > Chithra > OneDrive > python codes from 51 video > error.py > numbers  
1 numbers=[1,2,3]  
2 print(numbers[5])
```

Comment 1 of 1

Code Review

Attempting to access index 5 of the `numbers` list will raise an `IndexError` because the list only contains three elements (indices 0, 1, 2).
No change found to suggest.

Apply Discard

Explanation:

The program causes an error because it tries to access an index that does not exist in the list.

The list `numbers = [1, 2, 3]` has valid indices 0, 1, and 2 only.

Accessing `numbers[5]` is outside the list range, so Python raises an `IndexError`.

This is a runtime error that occurs during program execution.

AI identifies the issue as an out-of-range index access.

The error can be fixed by checking the list length before accessing an index.

Alternatively, a try-except block can be used for safe list access.

